

Anchoring Critic-Guided Fine-Tuning of Generative Robot Policies

Anonymous Authors

Abstract—Improving a pretrained robot policy requires balancing prior competence with new experience. The base policy provides useful but imperfect behavior; a learned critic suggests improvements but can be exploited by optimization. Relying too strongly on either can limit refinement or destroy existing skills. We present CAST (Constrained Action-Space Transport), a likelihood-free method for adapting generative robot policies with task-success rewards. CAST uses critic-guided action targets to train a residual over a frozen base. A soft restoring field permits local adaptation while increasingly resisting larger departures, and clipping limits each target correction. In a five-task LIBERO ablation, removing restoration causes complete collapse despite step clipping, showing that small updates alone do not preserve competence. Yet anchoring need not prevent improvement: per-task CAST fine-tuning gains 9.4 percentage points over the base on eight challenging LIBERO-90 tasks, outperforming the standard DICE-RL actor update, and over 50 points on robomimic square. Initial OpenArm evaluations show gains of 35–40 points across placement and stacking. These results support anchoring adaptation to prior competence as a practical principle for critic-guided policy refinement.

I. INTRODUCTION

For a robot that has already learned a skill, further learning should build on its competence rather than put that competence at risk. Generative policies trained on demonstrations offer a useful starting point, but they also inherit the limitations of their data. Reinforcement learning can help them improve through interaction without additional demonstrations [1], [2]. The difficulty is deciding how much to change: adhering too closely to pretrained behavior preserves its mistakes, while changing too freely can undo what already works. Policy refinement thus presents a stability–plasticity tension between retaining prior competence and adapting beyond it.

Critic-guided refinement makes this tension explicit through two imperfect sources of guidance. The pretrained policy encodes behavior supported by demonstrations, but cannot by itself say how to improve. A learned critic uses subsequent experience to predict which changes may help, but its predictions are only as reliable as the data supporting them. Optimizing those predictions can exploit errors outside well-covered regions, raising predicted return while reducing actual success [3], [4]. We study refinement with task-success rewards, where this failure is particularly costly: losing successful behavior also reduces the positive experience available to correct the critic. Neither the base nor the critic should therefore have unrestricted influence over the refined policy.

This leads to our central question: *how can a robot learn beyond its prior competence without allowing an uncertain improvement signal to erase that competence?* Residual fine-

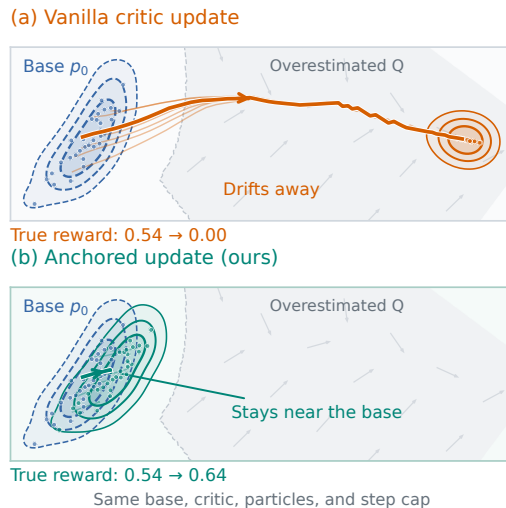


Fig. 1. **Same critic, different policy update.** Both panels start from the same blue base distribution and 384 paired actions. (a) Vanilla critic ascent moves probability mass into the overestimated region (gray). (b) Anchoring permits improvement near the base. Colored contours enclose 50/80/95% of the estimated mass; paths show representative updates in shared, rotated action coordinates. Scores are particle averages. This single-seed toy uses 25 critic-training samples, 60 updates, and a 0.06 step cap. Radius-0.15 per-particle projection illustrates CAST’s anchor, rather than full policy training.

tuning offers a natural setting: retain a frozen base and learn corrections guided by a critic [5]. Yet preserving the base’s parameters does not preserve the behavior of the combined policy; the residual can override the skill that remains stored in the base. Nor is a small update sufficient protection, since many small steps can accumulate into a large departure. Fig. 1 illustrates this distinction. We instead use the base as a continuing reference, with soft resistance that allows local adaptation and becomes stronger as departures grow.

We implement this principle with CAST (Constrained Action-Space Transport). Building on action-target training for generative policies [6], [7], CAST lets the critic propose changes to sampled actions and uses the frozen base to moderate those changes before the actor learns them. Each action is paired with its base counterpart under the same noise. A weak restoring pull acts throughout action space, with an additional term opposing departures beyond a chosen radius; clipping then caps the combined target displacement. The residual learns these detached targets by regression, without policy likelihoods or differentiation through the frozen generator. Restoration regulates accumulated departure, whereas clipping limits an individual requested correction. The radius controls this balance rather than certifying critic accuracy or guaranteeing the fitted policy’s behavior.

Our evaluation asks first whether anchoring leaves enough freedom to improve, then what makes that improvement possible. Controlled comparisons on robomimic and LIBERO hold the base, critic-training procedure, and interaction budget fixed, testing whether the actor update improves the use of the same learning resources. The gains are substantial: over 50 percentage points above the base on robomimic square and 9.4 points across eight LIBERO-90 tasks under per-task fine-tuning. The latter compares with 0.9 points for the standard DICE-RL actor. We extend this comparison to OpenArm placement and stacking, where success-only behavioral cloning (BC) tests whether simply imitating successful practice is sufficient. CAST gains 35–40 points over the base and has the highest observed success on both tasks, although its margins over the other refinements are small. Together, these studies test improvement beyond the base, efficiency relative to the standard actor, and the value of critic-guided refinement on physical skills.

The ablations then distinguish improvement from its possible explanations. On five LIBERO tasks, clipping without restoration yields zero success throughout, whereas their combination improves on every task’s base. Comparisons with alternative constraints and critic-value fields examine how much depends on the form of restoration and the direction of change. Finally, scope tests challenge the assumptions behind the balance: DMC shows that sparse reward alone does not explain the need for anchoring, while transport and shared-critic multitask results show that a fixed-base reference can become a liability. This progression establishes both the value and the limits of the principle: prior competence can support adaptation, but neither the base nor the critic is a universally reliable guide.

Our contributions are:

- A controlled study of competence-preserving policy refinement, distinguishing limits on individual updates from resistance to accumulated departure from a pre-trained policy.
- CAST, a likelihood-free residual update combining a bounded target correction with base restoration, supported by local-gradient and distribution-coupling analyses.
- Evaluations on robomimic, LIBERO, and DMC, together with a four-way real-robot refinement comparison on OpenArm placement and stacking.

II. RELATED WORK

A. Fine-Tuning Generative Robot Policies

Generative policies require specialized reinforcement learning updates because their action likelihoods are often intractable. DPPO optimizes a diffusion policy through its denoising process [1], while ReinFlow introduces stochastic transitions into flow sampling to enable policy-gradient updates [2]. Flow Matching Policy Gradients uses differences in flow-matching losses to construct a policy-update surrogate [8]. These methods exploit the structure of the generator or its training objective. In contrast, CAST needs only

samples from a frozen noise-to-action map and a critic. This makes its residual update applicable to one-step generators as well as iterative samplers. Our focus is how to preserve pretrained behavior during critic-guided improvement.

B. Critic-Guided Action Improvement

Using a critic to improve actions before fitting a policy is an established approach. DIPO modifies replay-buffer actions by gradient ascent and trains a diffusion policy on the resulting targets [6]; Q-Score Matching trains a diffusion policy using the critic’s action gradient [7]. Particle-based methods provide another connection: Soft Q-Learning trains an amortized sampler using Stein variational updates [9], and drifting models train one-step generators by fitting displaced samples [10]. Drifting Field Policy uses critic-ranked candidates to guide updates on a drifting backbone [11]. CAST builds on these ideas by constructing fresh action targets for a residual on a frozen base, with displacement clipping and explicit restoration toward the base. Stored transitions are left unchanged.

An alternative is to use critic values without following their action gradients. AWR and MPO favor actions through advantage- or value-weighted fitting [12], [13], while IDQL combines a diffusion behavior model with critic-based action selection [14]. These approaches motivate our ranking-only and value-weighted field variants. We compare them with the gradient field to distinguish the value of the improvement signal from the effect of anchoring.

C. Residual Learning and Pretrained-Policy Anchoring

Residual learning preserves a base controller while learning corrections. DICE-RL applies this construction to generative policies, combining a critic-driven residual actor with behavior-cloning regularization [5]. It is our closest system-level reference and the source of our controlled actor comparison. DAWN studies the critic cold-start and scaling issues that affect residual RL [15]. More broadly, TD3 and BCQ show how function-approximation and extrapolation errors can undermine critic-guided optimization [3], [4].

Our contribution is a controlled study of the restoring constraint: limiting each displacement, pulling toward a fixed base, and allowing a region of small corrections have different effects. A hinge can express the same radial geometry in a conventional actor loss, but clipping and finite optimization can yield different behavior. Our paired ablations therefore test both the geometry and its implementation across tasks. We do not assume that target regression alone resolves critic error.

III. PROBLEM STATEMENT

A. Setting

We consider online fine-tuning of a generative robot policy pretrained from demonstrations. An observation s includes the inputs used by the policy, and an action $a \in \mathbb{R}^{Hd}$ is a chunk of H controls with d dimensions each. A frozen

generator π_0 maps s and noise z to an action. Following residual RL [5], we learn a correction:

$$a_\theta(s, z) = \pi_0(s, z) + r_\theta(s, z), \quad z \sim \mathcal{N}(0, I). \quad (1)$$

An ensemble critic estimates return from replayed experience. Reward is provided on task success. The goal is to increase success through online interaction while retaining useful pretrained behavior. Our controlled comparison varies the actor update while matching the base, critic-training procedure, and interaction budget.

B. The Problem: Accumulated Departure

A critic can assign high values to actions outside its training coverage. Repeatedly following those predictions may move the policy away from actions that actually succeed [4]. A step cap alone does not prevent this: $\|a_{t+1} - a_t\| \leq \delta$ permits distance from the initial action to grow with the number of updates. Here t indexes optimization steps, not control steps. Fig. 1 compares two updates from the same base distribution under the same critic. Vanilla ascent moves probability mass into an overestimated region; anchoring each particle to its own base action permits improvement nearby.

The design question is how to permit useful corrections without allowing uncontrolled departure from the frozen base. We measure this departure by the residual’s root-mean-square action magnitude, $D_{\text{base}} = \mathbb{E}_{s,z}[\|r_\theta(s, z)\|_2 / \sqrt{Hd}]$, with states sampled from replay. A fixed-base restoring term acts on this departure, whereas step clipping limits only each proposed displacement. We study both empirically: bounded regression targets do not themselves guarantee bounded outputs after fitting a shared neural network.

IV. METHOD

CAST constrains the actions the actor is asked to learn. We adopt DICE-RL’s frozen-base residual architecture and off-policy training loop [5], replacing its actor objective with regression onto corrected action targets. Fig. 2 follows one update: sample current actions, displace them using the critic, apply restoration and clipping, and fit the residual to the resulting targets. Algorithm 1 places these steps in the online learning loop. All action-space operations use normalized coordinates, with dimension $D = Hd$.

A. Sample and Improve Actions

Step 1: sample (Fig. 2, upper left). For each replay state s , draw K noise samples. Pair each base action $a_{0,i} = \pi_0(s, z_i)$ with its current residual:

$$\bar{r}_i = \text{sg}[r_\theta(s, z_i)], \quad a_i = \pi_0(s, z_i) + \bar{r}_i, \quad (2)$$

where sg denotes stop-gradient. The blue base and gray current-actor distributions in the figure remain fixed during target construction; each correction refers to the base action with the same noise.

Step 2: improve (upper right). Let $\bar{Q}_\phi(s, z, a)$ be the conservative ensemble aggregate, omitting z when the critic does not use it. Normalize its action gradient by the detached

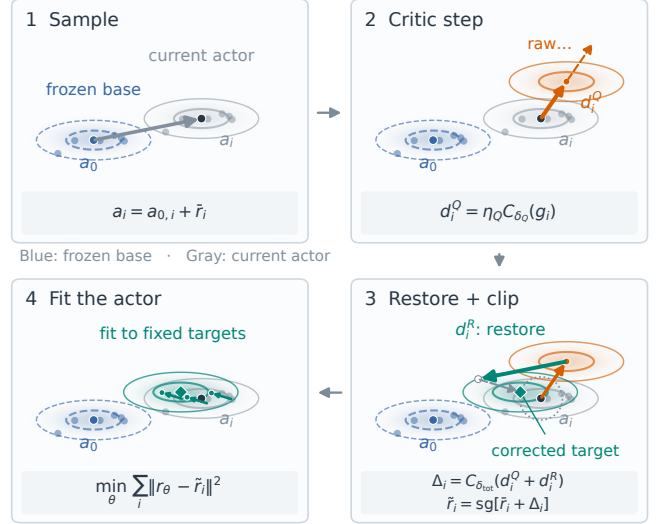


Fig. 2. **One coherent CAST update, read clockwise.** The blue base and gray current-actor distributions are identical in all four panels. The orange proposal from step 2 carries into step 3; the corrected green targets carry into step 4 for regression. Here $a_{0,i} = \pi_0(s, z_i)$ and $g_i = \nabla_a \bar{Q}_\phi / q_{\text{scale}}$. This constructed schematic follows (2)–(6) with illustrative parameters and a shared coordinate scale. The dashed raw request in step 2 continues off-panel. The green cloud denotes fixed targets, not a guaranteed post-update policy.

scale $q_{\text{scale}} = \max(\epsilon, \mathbb{E}_{\text{batch}}[\|\bar{Q}_\phi\|])$, computed over the current candidates. The orange displacement is

$$d_i^Q = \eta_Q C_{\delta_Q} \left(\frac{\nabla_a \bar{Q}_\phi(s, z_i, a_i)}{q_{\text{scale}}} \right), \quad (3)$$

where $C_\delta(v) = v \min(1, \delta / \|v\|_2)$, with $C_\delta(0) = 0$. The solid orange arrow is the accepted displacement; the dashed continuation shows the critic’s larger request. Clipping preserves small gradients and rescales large ones before applying η_Q . We differentiate the critic with respect to actions only, holding its parameters fixed for this step.

B. Constrain the Target

Step 3: restore and clip (lower right). A small step can still accumulate into a large departure. We therefore measure each candidate’s distance from its frozen reference in rms units, $m_i = \|\bar{r}_i\|_2 / \sqrt{D}$, and add the restoring displacement

$$d_i^R = -\eta_0 \bar{r}_i - \eta_{\text{anc}} \left(1 - \frac{\rho}{\max(m_i, \epsilon)} \right)_+ \bar{r}_i, \quad (4)$$

where $(x)_+ = \max(x, 0)$. The weak first term always pulls toward the base. The second permits a dead zone of radius ρ and opposes only the excess departure beyond it. The rms definition keeps the radius comparable across chunk dimensions. Unlike a reference to the preceding iterate, the frozen base does not move as training progresses.

The green arrow in Fig. 2 adds d_i^R head-to-tail to the orange proposal. This visual translation represents vector addition: the restoring field is evaluated at $\bar{r}_i$, not at the displaced action. We then cap the *combined* displacement

and detach the target:

$$\tilde{r}_i = \text{sg}\left[\bar{r}_i + C_{\delta_{\text{tot}}}(d_i^Q + d_i^R)\right]. \quad (5)$$

The green diamond lies within δ_{tot} of the current action, as indicated by the dotted circle. Thus $\|\tilde{r}_i - \bar{r}_i\|_2 \leq \delta_{\text{tot}}$. The two controls serve different purposes: restoration opposes accumulated departure from the base, while clipping limits the correction requested in one update. Restoration is soft; it does not project the target into the base-centered dead zone.

C. Fit the Residual

Step 4: regress (lower left). The corrected green cloud from step 3 becomes a set of fixed targets. We fit the residual network with mean-squared error:

$$\mathcal{L}_{\text{actor}} = \mathbb{E}_s \left[\frac{1}{KD} \sum_{i=1}^K \|r_\theta(s, z_i) - \tilde{r}_i\|_2^2 \right]. \quad (6)$$

Only this loss updates the actor. The base remains frozen, and the detached targets prevent the actor loss from differentiating through the critic. Freshly constructed targets leave executed replay transitions intact. The method needs no policy likelihood; deployment uses $\pi_0(s, z) + r_\theta(s, z)$.

Local gradient interpretation. The regression form changes how critic guidance is processed before it reaches the actor. The following identity makes that distinction precise.

Proposition 1: Fix the sampled states and latents, critic, and normalization scale at θ_{old} . Let $\Delta_i = \tilde{r}_i - \bar{r}_i$ and $J_i = \partial r_\theta(s, z_i) / \partial \theta$ at these parameters. Then

$$\nabla_{\theta} \mathcal{L}_{\text{actor}}|_{\theta_{\text{old}}} = -\frac{2}{KD} \mathbb{E}_s \left[\sum_i J_i^\top \Delta_i \right]. \quad (7)$$

Proof. The targets are detached. Differentiating (6) therefore gives $2J_i^\top (r_\theta - \tilde{r}_i)$ per sample; at the construction point, $r_\theta - \tilde{r}_i = -\Delta_i$. $\square$

For $\rho = 0$ or $\rho \geq \varepsilon$, the restoring field is the negative gradient of

$$U(r) = \frac{\eta_0}{2} \|r\|_2^2 + \frac{\eta_{\text{anc}}}{2} (\|r\|_2 - \sqrt{D\rho})_+^2, \quad (8)$$

with the usual continuous extension at zero. With both clips inactive, the local actor gradient is $2/D$ times that of the sampled objective with per-action term $-\eta_Q \bar{Q}_\phi / q_{\text{scale}} + U(r_\theta)$. Target regression alone does not supply a new first-order improvement direction. CAST’s explicit field and total-step clips rescale proposed displacements sample by sample before they pass through J_i . This also explains why a loss-space hinge is a meaningful control, without implying equivalence of the finite, clipped training procedures.

Distribution interpretation and limits. Let P_θ^s and P_0^s be the action distributions induced by the residual policy and frozen base at state s , with finite second moments. Sharing the latent z defines a valid coupling, so

$$W_2^2(P_\theta^s, P_0^s) \leq \mathbb{E}_z \|r_\theta(s, z)\|_2^2. \quad (9)$$

Here W_2 is the 2-Wasserstein distance. The bound follows by evaluating the transport cost under that coupling rather

Algorithm 1 CAST residual fine-tuning

Require: Frozen base π_0 , demonstrations $\mathcal{D}$, particles K , field and optimization settings

- 1: Initialize r_θ, Q_ϕ ; set $\phi^- \leftarrow \phi$
 - 2: Seed online replay $\mathcal{R}$ with base-policy rollouts
 - 3: **for** each online iteration **do**
 - 4: Collect rollouts using the baseline’s warm-up and value-guided exploration; append transitions to $\mathcal{R}$
 - 5: **for** each gradient step **do**
 - 6: Sample a minibatch from $\mathcal{R}$ and $\mathcal{D}$
 - 7: Form the multi-sample n -step TD loss using Q_ϕ
 - 8: **if** an actor update is due **then**
 - 9: Draw K noises per state; form $a_i, \bar{r}_i$ via (2)
 - 10: Form normalized critic steps d_i^Q via (3)
 - 11: Evaluate d_i^R at $\bar{r}_i$ via (4)
 - 12: $\tilde{r}_i \leftarrow \text{sg}[\bar{r}_i + C_{\delta_{\text{tot}}}(d_i^Q + d_i^R)]$
 - 13: Update θ by regression via (6)
 - 14: **end if**
 - 15: Update ϕ with the critic TD loss
 - 16: Polyak-update ϕ^- when scheduled
 - 17: **end for**
 - 18: **end for**
 - 19: **return** $\pi_\theta(s, z) = \pi_0(s, z) + r_\theta(s, z)$
-

than minimizing over all couplings. It connects residual control to distributional proximity without requiring a policy likelihood.

Neither identity proves that the learned actor stays inside a fixed ball. The target cap controls Δ_i , while the realized output change after a parameter step also depends on Jacobian coupling between samples. Likewise, (9) depends on the *actual* residual, not merely the requested target step. A nearby policy can still exploit a wrong critic direction, and a useful policy can lie farther from the base. Our experiments therefore test stability and improvement empirically.

V. EXPERIMENTS

We organize the evaluation around three questions. First, can anchoring preserve room for improvement beyond the pretrained policy, in simulation and on a physical robot? Second, which aspects of the update account for that behavior: restoration, clipping, or the critic’s improvement signal? Third, when does the balance cease to help? We establish refinement gains on robomimic, LIBERO, and OpenArm before using ablations to examine the mechanism and broader control experiments to identify its limits.

A. Setup and Reporting

We use state-based robomimic manipulation [16], image-based tasks from LIBERO-90 [17], and the DeepMind Control Suite (DMC) [18]. Our controlled actor comparisons share the pretrained checkpoint, critic-training procedure, replay recipe, and interaction budget. The DICE-RL actor maximizes normalized critic value with behavior-cloning regularization; CAST replaces that actor update. Both retain the frozen observation encoder where applicable, ensemble TD learning, multi-step returns, and demonstration replay.

Robomimic uses 300-episode evaluations. The drifting-base comparison reports last-three-checkpoint averages over

TABLE I

ROBOMIMIC SUCCESS. TOP: COMPARISONS WITHIN EACH BASE. DRIFTING ROWS SHOW THREE-SEED RANGES FROM 300-EPIISODE EVALUATIONS, AVERAGED OVER THE LAST THREE CHECKPOINTS; † TWO CAN BASELINES ENDED BEFORE THE 20K UPDATE BUDGET. THE DIFFUSION PAIR IS AT 496K ENVIRONMENT STEPS; z DENOTES A VALUE-ONLY CAST FIELD. BOTTOM: EXTERNAL SQUARE REFERENCES, WITH THEIR OWN EVALUATION BASELINES. * USES A STRICTER SUCCESS RULE; ° IS AN ONGOING RUN. BUDGETS AND PROTOCOLS DIFFER BETWEEN BLOCKS. BOLD MARKS THE BEST RESULT PER TOP ROW AND THE BEST FINE-TUNED SCORE AND GAIN WITHIN EACH EXTERNAL EVALUATION-BASELINE GROUP.

Task / base class	Base	DICE-RL	+ CAST
Can / drifting	.877	.940–.970 [†]	.978–.993
Square / drifting	.382	.861–.909	.912–.930
Square / diffusion	.600	.965	.959
Square / flow matching	.570	.831	.934^z
<i>External square references (own harness and budget)</i>			
Method	Own base	Fine-tuned	Δ
DIPO [6]*	.348	.156	−.192
QSM [7]*	.348	.000	−.348
DQL*	.348	.000	−.348
IDQL [14]*	.348	.327	−.021
AWR [12]*	.348	.393	+ .045
DPPO [1]	.600	.678[°]	+ .078

three training seeds. LIBERO uses 100 rollouts $\times$ three evaluation seeds, which should not be confused with independent training runs. DMC reports return over 1000-step episodes, averaged over ten evaluation episodes. Training-seed counts and unfinished runs are marked with their tables. All improvements are relative to each method’s own base and evaluation rule.

The LIBERO recipe uses $K = 16$, $\eta_0 = 0.05$, $\eta_{\text{anc}} = 1$, $\rho = 0.05$ rms, $\delta_Q = 1$, and $\delta_{\text{tot}} = 0.15$. Reward steps $\eta_Q \in \{1, 2\}$ are shared across tasks. Robomimic and the default DMC transfer use $\eta_Q = 0.5$ and $\rho = 0$; the tuned DMC quadruped setting is disclosed below. The target-construction rule is unchanged across these settings.

B. Refinement in Simulation

Anchoring is useful only if it allows the robot to improve beyond its starting behavior. We test both the size of that improvement and the interaction required to obtain it, then examine whether it persists across visual tasks and stronger bases.

Robomimic: gains at a fixed budget. The drifting-base rows of Table I isolate the actor update. On square, CAST reaches .912–.930 from a .382 base, compared with .861–.909 for backpropagation through the critic. It leads on each training seed at the reported 20K update budget. Can provides a near-saturation check: CAST reaches .978–.993 from .877, although two baseline seeds end before the full budget and are marked accordingly.

Sample efficiency and generator choice. The gain is partly one of efficiency. With longer training on drifting square, both actors approach .97–.99 near 40K updates. On

the retrained diffusion base, they also reach similar endpoints (.965 for DICE-RL and .959 for CAST at 496K environment steps). CAST first reaches .90 at 128/160/160K steps, compared with 224/240/256K for DICE-RL: about one-third fewer environment steps on average. On the flow-matching base, the value-only CAST variant reaches .934 against .831. This supports portability of the update framework, without implying that one reward-field choice is optimal for every generator.

Context from other fine-tuners. The lower block provides external context. DIPO, QSM, DQL, IDQL, and AWR use the DPPO authors’ adapted implementations and approximately 24M environment steps. Their stricter chunk-level success rule reads the shared checkpoint at .348 rather than .600; we therefore use .348 for their own-base deltas. QSM and DQL collapse in these runs, whereas AWR gains .045. These heterogeneous systems are not a matched actor-update test. DPPO is still training on the .600 base; a separate converged run on its original-architecture base reaches .887 at 11.7M steps.

LIBERO: transfer across visual manipulation tasks.

The next comparison asks whether the update helps beyond a single skill. We fine-tune a separate residual and critic for each of eight challenging LIBERO-90 tasks, starting from one multitask base and routing by task ID at deployment. Table II reports both shared reward-step settings rather than choosing the best setting independently per task. On the drifting base, CAST₂ improves mean success from .661 to .755 (+9.4 points), compared with .670 (+0.9) for the backpropagated actor. CAST improves on the base on all eight tasks, although DICE-RL is better on the Study-3 caddy task (.653 versus .635).

A stronger base changes the room for improvement.

The stronger flow-matching base leaves a smaller aggregate gain. CAST₁ is nearly unchanged from the base (.756 versus .753), but CAST₂ reaches .777, while DICE-RL averages .750. The result therefore depends on the reward step as well as the base. Gains are not universal: on some tasks the unmodified flow-matching policy remains best. A critic-free GRPO comparison is available on four tasks under the matched evaluation protocol; it is competitive on red mug (.883 versus CAST₁’s .878), while CAST leads on ketchup (.738 versus .700). We do not pool the four additional short-run GRPO estimates into a matched eight-task mean.

C. Refinement on the Physical Robot

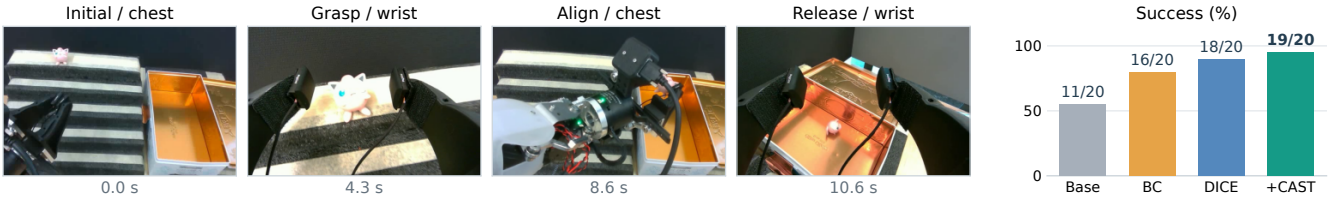
The simulation comparisons isolate the actor update; the physical study asks whether it adds value when refining an existing robot skill through practice. Success-only BC provides a critic-free alternative, separating the benefit of imitating successful attempts from that of learning from both outcomes. We use an OpenArm platform’s left seven-joint arm and gripper, with chest and left-wrist RGB cameras, measured joint and gripper positions, and a fixed task-language prompt. The two tasks are *Jigglypuff into the case* and *red-on-purple stacking*. Both use a hand-made staircase: different support heights and nearby edges make approach,

TABLE II

LIBERO HARD-8, PER-TASK FINE-TUNING WITH TASK-ID ROUTING. SUCCESS IS EVALUATED WITH 100 ROLLOUTS $\times$ 3 EVALUATION SEEDS. CAST₁ AND CAST₂ USE SHARED REWARD STEPS $\eta_Q = 1$ AND 2, RESPECTIVELY, ACROSS ALL EIGHT TASKS. EACH BLOCK STARTS FROM ITS OWN PRETRAINED MULTITASK BASE. COLUMN MEANS ARE FIXED-RECIPE AVERAGES, NOT PER-TASK BEST-OF SELECTIONS. BOLD MARKS THE BEST VALUE IN EACH ROW WITHIN EACH BASE BLOCK.

Task	Drifting base				Flow-matching base			
	Base	DICE	CAST ₁	CAST ₂	Base	DICE	CAST ₁	CAST ₂
Drawer / bowl (8)	.818	.820	.831	.857	.897	.873	.900	.887
Stove / pan (21)	.690	.703	.737	.752	.700	.730	.690	.727
Drawer / ketchup (32)	.610	.587	.694	.729	.690	.682	.738	.773
Basket / juice (53)	.785	.800	.842	.853	.917	.915	.897	.913
Plate / red mug (65)	.573	.550	.826	.812	.787	.809	.878	.887
Front caddy / book (73)	.648	.663	.710	.752	.683	.690	.667	.783
Right caddy / book (75)	.578	.583	.657	.648	.620	.592	.608	.563
Study-3 caddy / book (81)	.588	.653	.607	.635	.733	.708	.670	.680
Mean	.661	.670	.738	.755	.753	.750	.756	.777

(a) Jigglypuff into the case



(b) Red cube onto the purple cube

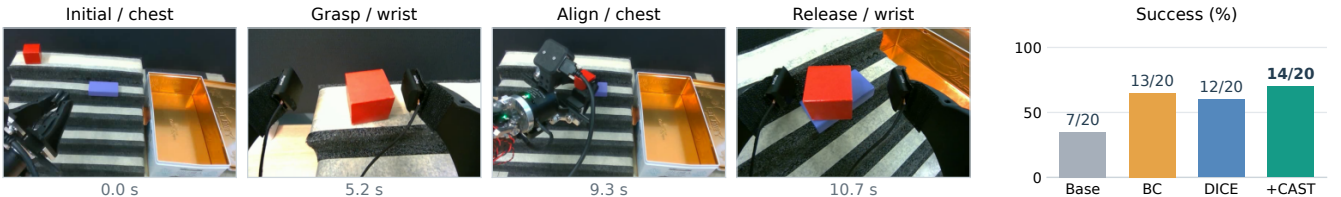


Fig. 3. OpenArm task demonstrations and autonomous evaluation results. Each row shows initial, grasp, align, and release stages, using chest, left-wrist, chest, and left-wrist views, respectively. These are successful *teleoperated demonstrations*, not autonomous evaluations of the compared methods. Times are nominal video times at 15 fps, using the source stage frame indices. A common background crop is used throughout. The fifth panel reports autonomous success over 20 trials per method for that row’s task, with successes/trials above the bars and a shared percentage scale. Base is the pretrained FM policy; BC is success-only refinement; DICE and +CAST are DICE-RL and DICE-RL + CAST. Bold counts mark the highest observed success. Jigglypuff poses are randomized on the stairs.

grasp alignment, and release important. Stacking additionally requires accurate placement before withdrawing the gripper.

Four-way comparison. For each task, a flow-matching policy is pretrained on teleoperated demonstrations. It predicts 30-step chunks of absolute joint and gripper targets at 15 Hz using a ten-step Euler sampler; deployment executes a prefix before replanning. All refinement conditions start from the same checkpoint. Success-only BC continues supervised training on successful online trajectories. DICE-RL and DICE-RL + CAST use both successful and failed attempts, with the same frozen base, critic architecture, replay, exploration, and update schedule. This tests whether critic-guided refinement adds value beyond imitating successful practice, and whether the actor-update rule changes that benefit.

Collection and evaluation. After each attempt, an operator assigns the success/failure label. Valid failures are retained for the RL arms, and invalid or discarded trials are

accounted for explicitly. Replay records commands actually executed after chunk-prefix selection and command limiting. The refinement protocol uses matched robot interaction budgets and execution limits; evaluation attempts are held out from refinement data. Each method is evaluated over 20 rollouts per task. For Jigglypuff, the toy’s pose is randomly changed on the stairs between attempts. Exact pose ranges and paired resets across methods are not established by the available evaluation record. Toy placement requires release fully inside the case with the toy remaining there. Stacking requires the red cube to remain supported by the purple cube after release and gripper withdrawal; the post-release observation interval is not specified in the reported stacking results.

Results across both tasks. Fig. 3 reports 11/20 successes for the base FM policy, 16/20 after success-only BC refinement, 18/20 for DICE-RL, and 19/20 for DICE-RL + CAST

TABLE III

CONSTRAINT AND REWARD-FIELD ABLATIONS. (A) REPORTED THREE-SEED FIVE-TASK LIBERO FACTORIAL; MEAN BASE SUCCESS IS .673. (B) THE CONSTRAINT COMPARISON USES THREE TRAINING SEEDS, 300-EPIISODE SQUARE EVALUATIONS, AND POWERED LIBERO EVALUATIONS. (C) REWARD-FIELD SUBSTITUTIONS, WITH SQUARE SEED RANGES. EACH STUDY BLOCK RETAINS ITS OWN REPORTED CAST REFERENCE; VALUES ARE NOT POOLED ACROSS BLOCKS. MUG AND KETCHUP DENOTE TASKS 65 AND 32. BOLD MARKS THE BEST RESULT PER COLUMN WITHIN EACH STUDY BLOCK.

<i>(a) Step clipping versus base restoration</i>			
Step clip	Restoration	Five-task mean	
Off	Off	.000	
On	Off	.000	
Off	On	.657	
On	On (full CAST)	.703	

<i>(b) Alternative constraint implementations</i>			
Constraint	Square	Mug	Ketchup
Loss-space hinge	.28	.77	.65
Hard target projection	.96	.93	.54
Full CAST	.94	.78	.68

<i>(c) Reward field, with the constraint fixed</i>			
Field	Square	Mug	Ketchup
Gradient	.912–.930	.600	.627
Top- k	.525–.562	.577	.341
Value-tilted	.560–.584	.697	.378

on Jigglypuff. On stacking, the same methods achieve 7/20, 13/20, 12/20, and 14/20 successes. All refinements improve observed success over the base on both tasks. CAST reaches the highest observed rates, 95% and 70%, gaining eight and seven successes over the bases. The comparison with BC is task-dependent: DICE-RL exceeds BC on Jigglypuff but trails it on stacking. CAST exceeds DICE-RL by one and two trials, and BC by three and one. These small margins support further evaluation rather than a reliable ranking of refinement methods. Fig. 3 illustrates both tasks using separate demonstration sequences.

D. What Makes the Balance Work?

The refinement comparisons establish that constrained updates can improve pretrained skills. We now separate three possible explanations: limiting each step, resisting accumulated departure, and choosing an informative direction of change. These are controlled interventions on the update, rather than direct measurements of critic reliability.

Clipping and restoration have different roles. Table III(a) switches the two independently. Disabling clipping removes both field and total-step caps; disabling restoration sets both η_0 and η_{anc} to zero. Without restoration, success is zero on every factorial task, even with clipping. Restoration alone avoids complete collapse but is inconsistent: red mug improves from .573 to .810, whereas ketchup falls from .610 to .387. The full pair is the only arm above its base on all five tasks, with mean success .703 against .673 before fine-tuning. This is direct evidence that a per-step cap and a fixed-base restoring term are not interchangeable.

TABLE IV

DMC EPISODE RETURN (MAXIMUM 1000). ALL ACTORS USE THE SAME FROZEN BASE PER TASK. SCORES AVERAGE 10 EVALUATION EPISODES; THE TARGET BUDGET IS 400K ENVIRONMENT STEPS. $\circ$ DENOTES A BELOW-BUDGET RUN, ν HIGH EVALUATION VARIANCE, AND t A TUNED REWARD STEP. SWING-UP AVERAGES TWO TRAINING SEEDS; THE OTHER ROWS CURRENTLY HAVE ONE, WITH SECOND SEEDS PENDING. BOLD MARKS THE HIGHEST REPORTED RETURN PER ROW, INCLUDING TIES AND INTERIM RUNS.

Task	Base	No constraint	Backprop	CAST
<i>Dense reward</i>				
Cartpole swing-up	429	575	318	715
Cartpole balance	378	815 $^\circ$	844	911
Quadruped run	238	97 $^\circ$	283	312^{of}
Finger spin	3	0 $^\circ$	187°	187°
<i>Sparse reward</i>				
Cartpole balance	90	880 $^\circ$	843 $^\nu$	1000
Ball-in-cup catch	86	940°	66 $^\circ$	925

Does the form of restoration matter? Block (b) compares the field update with a dead-zone hinge in the DICE-RL loss and a hard projection of the action target. The hinge performs well on the two LIBERO tasks but reaches only .28 on square, with reported training-seed outcomes spanning .03–.77. Hard projection is strongest on red mug and competitive on square, but falls below the ketchup base. CAST is consistently effective across these three tasks, rather than uniformly best. We retain each study block’s reported full-method reference instead of pooling aggregates across ablation campaigns.

Does improvement require the critic’s gradient? Block (c) replaces it with kernel transport toward top- k candidates or softmax-weighted standardized Q -values, holding the constraint fixed. Value-only fields lose substantially on square and ketchup. On red mug, however, the tilted field exceeds the gradient reference in this study (.697 versus .600). Thus the gradient is valuable but not universally superior; its usefulness depends on the critic and task.

How much freedom should the correction have? Sensitivity studies further show that an always-on pull can suppress useful corrections on LIBERO: $\rho = 0$ yields .549/.601 on mug/ketchup. Interior radii from .02 to .20 give a broad operating range, while making the dead zone inert still retains the weak pull. Single-seed radius and step-size differences should be interpreted cautiously, particularly on red mug, where substantial training-seed variation has been observed.

E. Scope and Failure Cases

The preceding results support anchoring in the tested manipulation settings. They leave open whether sparse rewards alone explain its value, and whether the base always provides a useful reference.

DMC: reward sparsity is not sufficient to explain the gains. DMC tests the same residual update outside the main manipulation setting. Bases are one-step policies distilled from SAC experience. Table IV retains all six reported tasks and marks incomplete runs rather than treating them as

converged comparisons. CAST raises dense swing-up return from 429 to 715, versus 318 for the backpropagated actor, and dense balance from 378 to 911. Quadruped run reports 312 before the matched budget using $\eta_Q = 1$, compared with 267 for the untuned CAST arm and 283 for backpropagation. This is a tuned, interim comparison, not a converged matched-budget advantage or evidence of one optimal recipe.

On sparse balance, CAST reaches the maximum return of 1000 from a base of 90; on ball-in-cup catch it reaches 925 from 86. Unconstrained updates can also improve in this setting, including under sparse reward, so sparsity alone does not explain the manipulation failures. Conversely, dense reward does not ensure useful unconstrained fine-tuning: the incomplete quadruped run remains below its base. These observations are consistent with a role for critic error, data coverage, and departure from the base, but do not isolate their individual effects. The DMC recipe’s $\rho = 0$ also shows that a free dead-zone interior is not necessary in every regime.

When the reference or critic changes, the ordering can reverse. On robomimic transport, CAST reaches .594 against .912 for DICE-RL at a matched 2.24M environment steps. This failure is consistent with anchoring restricting a needed large correction from a weak base. With a shared multitask LIBERO critic, best-of-16 evaluation gives .722 for DICE-RL and .692 for CAST, reversing the per-task ordering. Together, these cases show why neither proximity to the base nor one actor-update rule is sufficient for reliable improvement. The useful balance depends on the starting behavior and the critic used to guide adaptation.

VI. DISCUSSION AND CONCLUSION

CAST combines critic-guided action improvement with a restoring reference to the pretrained policy and a cap on the target displacement. Its local gradient interpretation connects it to regularized actor learning, while its explicit targets make the action-space correction available before fitting the actor. The robomimic and LIBERO comparisons show useful gains at controlled budgets, and the constraint factorial demonstrates that small steps alone can still destroy pretrained behavior. Alternative constraints can be stronger on individual tasks; the evidence favors consistent refinement rather than universal superiority of one update. On OpenArm, all three refinements improve observed success over the base on both Jigglypuff placement and stacking. CAST achieves 19/20 and 14/20 successes, respectively, versus base counts of 11/20 and 7/20. Its margins over DICE-RL are one and two trials, while BC outperforms DICE-RL on stacking. These results add physical evidence for refinement, with larger evaluations needed to resolve differences between update rules.

The failure cases in Sec. V-E show that anchoring can be counterproductive with a weak reference or a shared multitask critic. DMC further shows that neither sparse reward nor a dead-zone interior alone determines which update works. A target bound is not a theorem about the learned policy’s outputs, and the radius is not derived from critic reliability. Adapting that radius to justified local trust is

a natural next step. Broader physical evaluation is needed to assess how these findings generalize across tasks and initial conditions.

REFERENCES

- [1] A. Z. Ren, J. Lidard, L. L. Ankile, A. Simeonov, P. Agrawal, A. Majumdar, B. Burchfiel, H. Dai, and M. Simchowitz, “Diffusion policy optimization,” in *International Conference on Learning Representations (ICLR)*, 2025, arXiv:2409.00588.
- [2] T. Zhang, C. Yu, S. Su, and Y. Wang, “ReinFlow: Fine-tuning flow matching policy with online reinforcement learning,” 2026. [Online]. Available: <https://arxiv.org/abs/2505.22094>
- [3] S. Fujimoto, H. van Hoof, and D. Meger, “Addressing function approximation error in actor-critic methods,” in *International Conference on Machine Learning (ICML)*, 2018.
- [4] S. Fujimoto, D. Meger, and D. Precup, “Off-policy deep reinforcement learning without exploration,” in *International Conference on Machine Learning (ICML)*, 2019.
- [5] Z. Sun and S. Song, “From prior to pro: Efficient skill mastery via distribution contractive RL finetuning,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.10263>
- [6] Z. Li, R. Krohn, T. Chen, A. Ajay, P. Agrawal, and G. Chalvatzaki, “Learning multimodal behaviors from scratch with diffusion policy gradient,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.00681>
- [7] M. Psenka, A. Escontrela, P. Abbeel, and Y. Ma, “Learning a diffusion model policy from rewards via Q-score matching,” 2025. [Online]. Available: <https://arxiv.org/abs/2312.11752>
- [8] D. McAllister, S. Ge, B. Yi, C. M. Kim, E. Weber, H. Choi, H. Feng, and A. Kanazawa, “Flow matching policy gradients,” 2025. [Online]. Available: <https://arxiv.org/abs/2507.21053>
- [9] T. Haarnoja, H. Tang, P. Abbeel, and S. Levine, “Reinforcement learning with deep energy-based policies,” in *International Conference on Machine Learning (ICML)*, 2017.
- [10] M. Deng, H. Li, T. Li, Y. Du, and K. He, “Generative modeling via drifting,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.04770>
- [11] J. Koo, M. Park, J. Choi, Y. Min, and M. Sung, “Drifting field policy: A one-step generative policy via wasserstein gradient flow,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.07727>
- [12] X. B. Peng, A. Kumar, G. Zhang, and S. Levine, “Advantage-weighted regression: Simple and scalable off-policy reinforcement learning,” *arXiv preprint arXiv:1910.00177*, 2019.
- [13] A. Abdolmaleki, J. T. Springenberg, Y. Tassa, R. Munos, N. Heess, and M. Riedmiller, “Maximum a posteriori policy optimisation,” in *International Conference on Learning Representations (ICLR)*, 2018.
- [14] P. Hansen-Estruch, I. Kostrikov, M. Janner, J. G. Kuba, and S. Levine, “IDQL: Implicit Q-learning as an actor-critic method with diffusion policies,” *arXiv preprint arXiv:2304.10573*, 2023.
- [15] G. Ma, L. Li, H. Wang, Z. Liu, P.-L. Bacon, and D. Tao, “What makes value learning efficient in residual reinforcement learning?” 2026. [Online]. Available: <https://arxiv.org/abs/2602.10539>
- [16] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín, “What matters in learning from offline human demonstrations for robot manipulation,” in *Conference on Robot Learning (CoRL)*, 2021.
- [17] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” 2023. [Online]. Available: <https://arxiv.org/abs/2306.03310>
- [18] Y. Tassa, Y. Doron, A. Muldal, T. Erez, Y. Li, D. d. L. Casas, D. Budden, A. Abdolmaleki, J. Merel, A. Lefrancq, T. Lillicrap, and M. Riedmiller, “DeepMind control suite,” *arXiv preprint arXiv:1801.00690*, 2018.